Scalable Services (SEZG583)

Scalable Services Group 23

Assignment 1

3 November 2026

SlotHub — Design and Technical Report

Assignment 1: Microservices Design and Development

#	Team member	BITS ID	Main contribution
1	Subhodip Banerjee	2025TM93191	Scheduling, Booking, shared building blocks, Challenges 1 and 3B
2	J K Shekar	2025TM93095	Payment, Notification, tests, Challenge 2 evidence
3	Ria Mishra	2025TM93010	Payment service, payment and idempotency tests
4	Harshit Pathak	2025TM93557	—

Links	
Repository	github.com/2025tm93191-stack/slothub
Documents and videos (Drive)	drive.google.com/drive/folders/1daM99AjyiiTwf_5zDP_LFsxPC_0au-pV
Combined team demo	**drive.google.com/file/d/13urxxFpP20OoY6Wk4k6CUl0AwoOm30vJ/view**
Demo: Subhodip Banerjee	drive.google.com/file/d/1zJubB6u2nK4iz1c44ELd6BJD6N7-QN4P/view
Demo: J K Shekar	drive.google.com/file/d/1jmBtugeWTK5HwaKVv9CwCKbT2H5AC1Ul/view
Demo: Ria Mishra	drive.google.com/file/d/1XXrJP860TmP7rJJmzIhfIMMfLspUbZXL/view

Stack: .NET 10, ASP.NET Core minimal APIs, EF Core 10, PostgreSQL 17, Polly, Serilog, xUnit, Testcontainers, Docker Compose

1. Problem and scope
2. Service decomposition
3. Architecture
4. Data design
5. API design
6. Engineering challenges
7. Testing
8. Design decisions and trade-offs
9. Limitations and future work
10. Links

Appendices: A Contributions · B Reflections · C AI-use declaration · D Business-rule traceability · E How to run

1. Problem and scope

SlotHub is a multi-tenant booking platform for any business that sells time, such as clinics, salons, sports turfs and tutors. A business (*tenant*) publishes slots for its resources (a doctor, a court). A customer holds a slot, pays, and receives an SMS/email confirmation. An unpaid hold is released automatically. The domain was chosen because it naturally contains every distributed-systems problem the brief asks for, and because one engine can later serve many industries as a product.

Assignment condition	How SlotHub meets it
(a) ≥ 4 services	Scheduling, Booking, Payment (mock gateway), Notification (mock SMS/email): four distinct responsibilities, four databases
(b) Transaction over ≥ 3 services	Book: Booking → Scheduling (hold) → Payment (charge) → Scheduling (confirm) → Notification (via outbox)
(c) Contended resource	Slot capacity (1 for a doctor, N for a class)
(d) Unsafe to repeat	Creating a booking, paying, cancelling, charging, refunding
(e) Slow / failing dependency	Mock payment gateway and SMS provider with runtime failure and latency injection (`PUT /api/v1/admin/chaos`)

Business rules

ID	Rule	Owner
BR1	A slot never has more active holds (HELD + CONFIRMED) than its capacity.	Scheduling
BR2	A hold expires after `HOLD_TTL_SECONDS` (30 s in the demo) if not confirmed and its capacity is returned; a confirmed hold is never expired.	Scheduling
BR3	A booking is confirmed only after a successful payment and while its hold is active; a payment that completes after the hold expired is refunded automatically.	Booking

BR4	The same `Idempotency-Key` with the same payload returns the original result with no new effect; with a different payload it is rejected (422).	Booking, Payment
BR5	Cancellation refund: ≥ 24 h before the slot 100 %, < 24 h 50 %, after start not allowed.	Booking
BR6	A slot that has already started cannot be held.	Scheduling

Assumptions. Fixed price per service offering, snapshotted onto the hold. Integer minor units (paise). UTC timestamps. Customers are identified by ID plus phone/email in the request. **Out of scope** (per brief): authentication, UI, real payment/SMS integration. Assignment 2 deployment (Minikube) is not part of this report; Dockerfiles exist only to run the stack locally.

2. Service decomposition

Service	Responsibility (one sentence)	Owns	Does not own
Scheduling :5101	Defines what a tenant offers and when, and controls how much of each slot is left.	Tenants, resources, offerings, prices, slots, holds; hold expiry	Bookings, money
Booking :5102	Takes a booking from request to confirmation or cancellation, orchestrating the others.	Bookings, state machine, status history, outbox, idempotency keys	Capacity, payments
Payment :5103	Charges and refunds money exactly once per request (mock gateway).	Payments, refunds, idempotency keys, chaos settings	Why money moved
Notification :5104	Delivers customer messages once and retries failed deliveries.	Messages, delivery attempts, templates, chaos settings	Booking state

Why these boundaries. We split by business capability and by rate of change. Catalog and availability are one service (Scheduling) because a service's duration defines how its slots are cut, so they always change together. Splitting them would create a distributed monolith and add a hop to every booking. Booking is the only orchestrator: it is the one place that knows the end-to-end flow and its compensations. The other services expose small, reusable, idempotent capabilities. Payment is separate because it is the slow/failing dependency and money needs its own audit trail and idempotency. Notification is separate because its failure must never affect a booking and its channels will multiply independently.

Alternatives rejected. (1) Separate Catalog and Availability services: they change together, and one more network hop. (2) Payment inside Booking: it loses the failing-dependency boundary and the second idempotency layer. (3) A Customer service: it would be a CRUD wrapper while authentication is out of scope. (4) Choreography with events only: the flow and its compensation would be scattered across services and harder to test (see D2).

3. Architecture

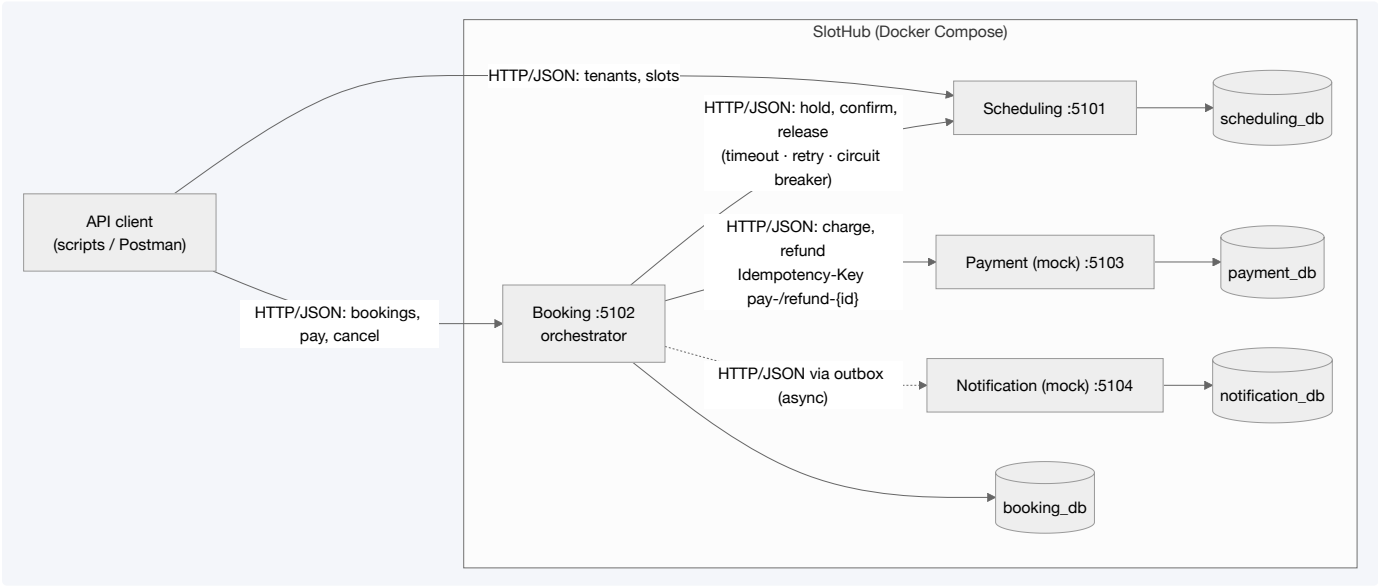


Figure 1 — Components. Solid arrows: synchronous REST calls (direction = caller → callee), all carrying `X-Correlation-Id`. Dotted: asynchronous delivery from Booking's transactional outbox. Each database has its own user; no service reads another's database.

The longest synchronous chain is **client → Booking → Scheduling or Payment** (depth 2). Booking calls downstream services one after another, never through each other. Every service exposes `GET /health` (liveness) and `/health/ready` (own database). It reads all configuration from environment variables and writes structured JSON logs carrying the correlation ID it received or generated. That ID is forwarded on every outgoing call. Inside each service: `Api` (HTTP, validation) → `Domain` (entities, state machines, rules; no framework dependencies) ← `Infrastructure` (EF Core, HTTP clients, background workers).

Main transaction: happy path

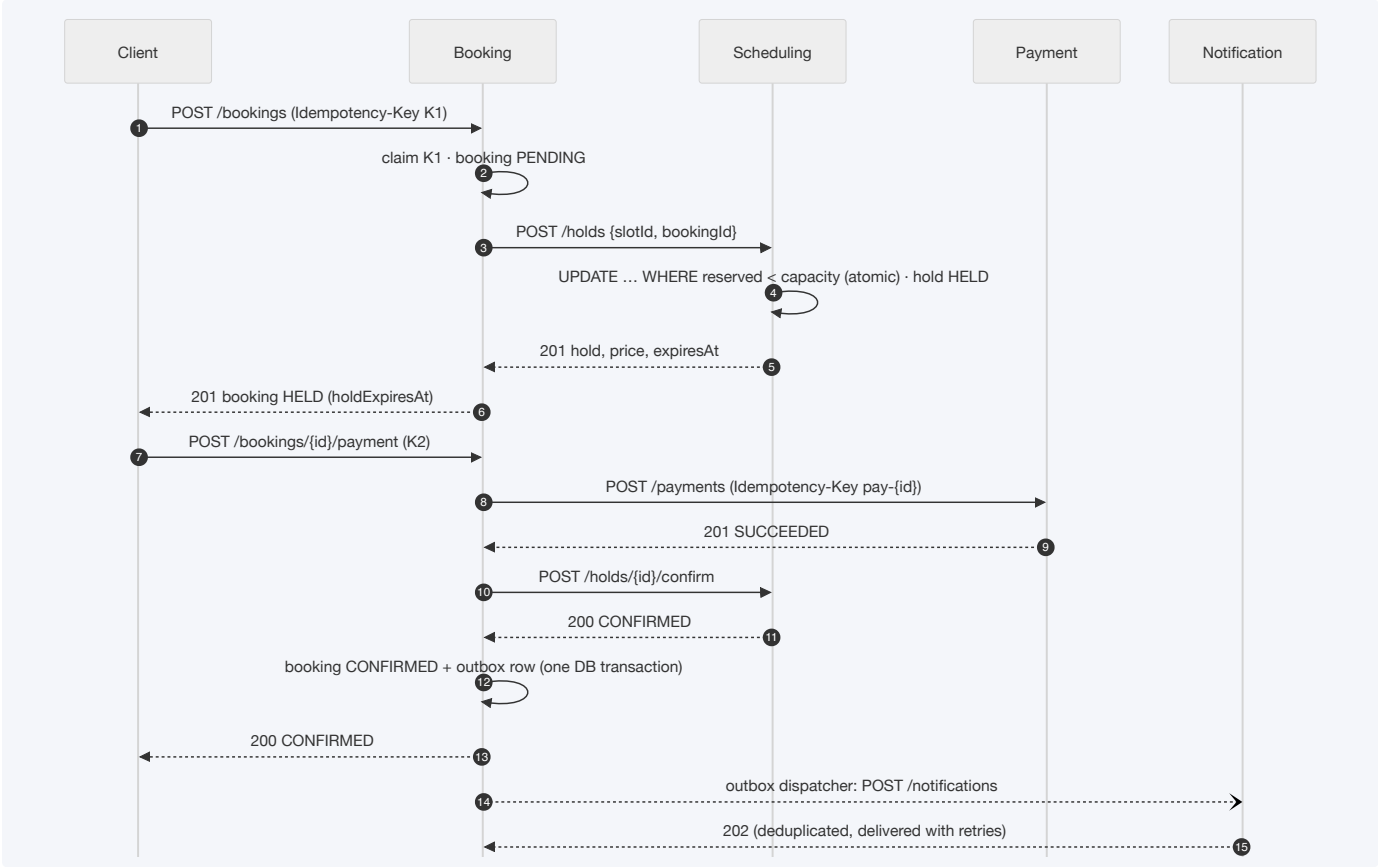


Figure 2 — Two-step checkout (hold, then pay), as on a real payment page; it gives the hold a real lifetime for Challenge 3B.

Main transaction: failure paths

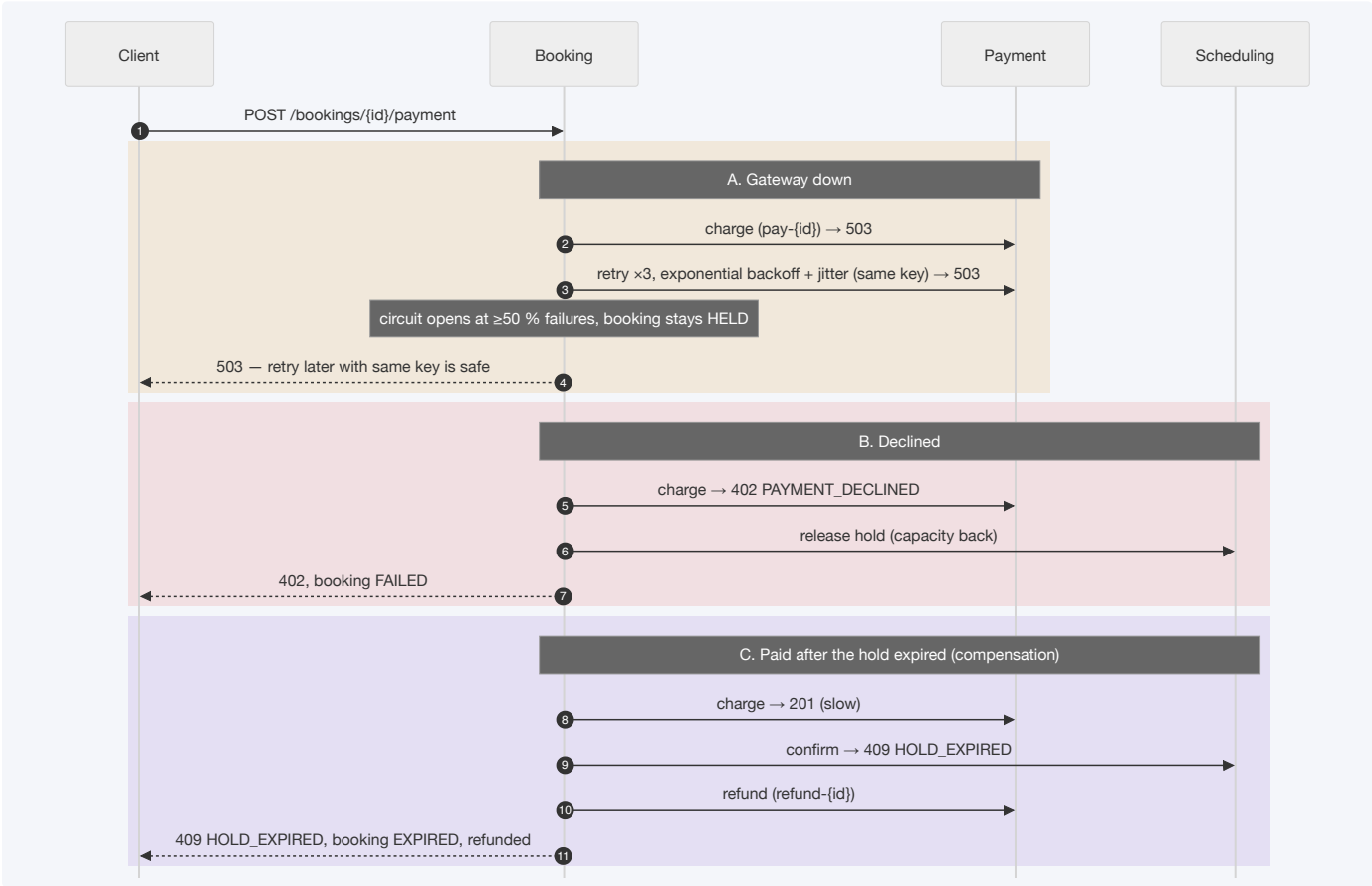
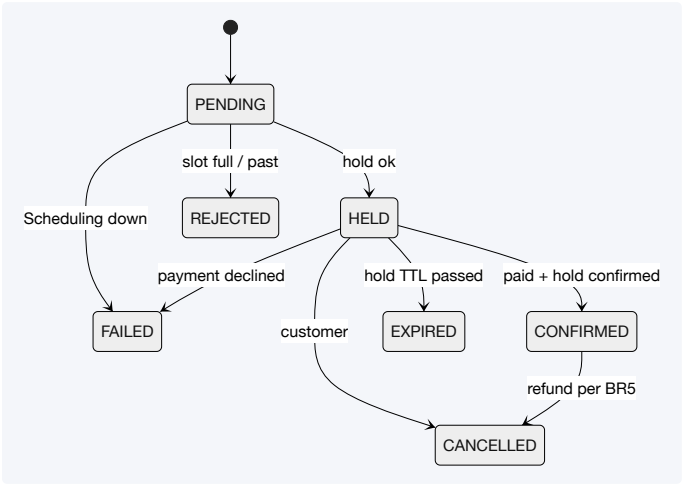
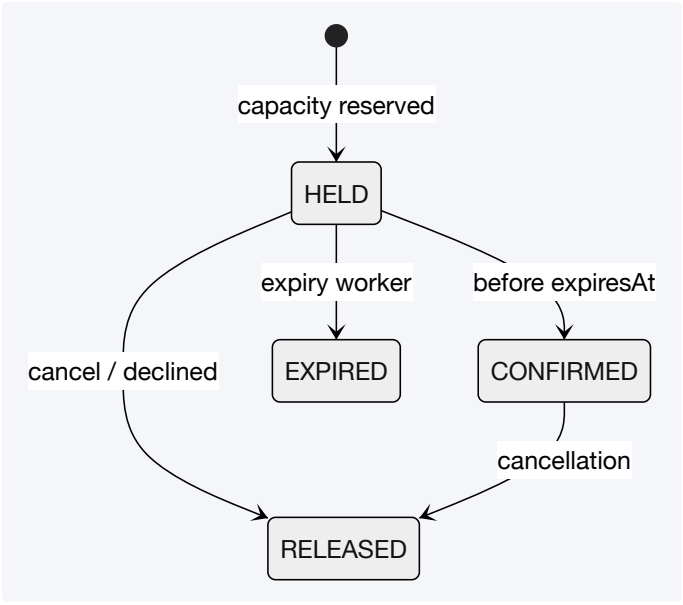


Figure 3 — Failure paths, all demonstrated by scripts (e2e sections 2–3, Challenge 3B section E).

State machines



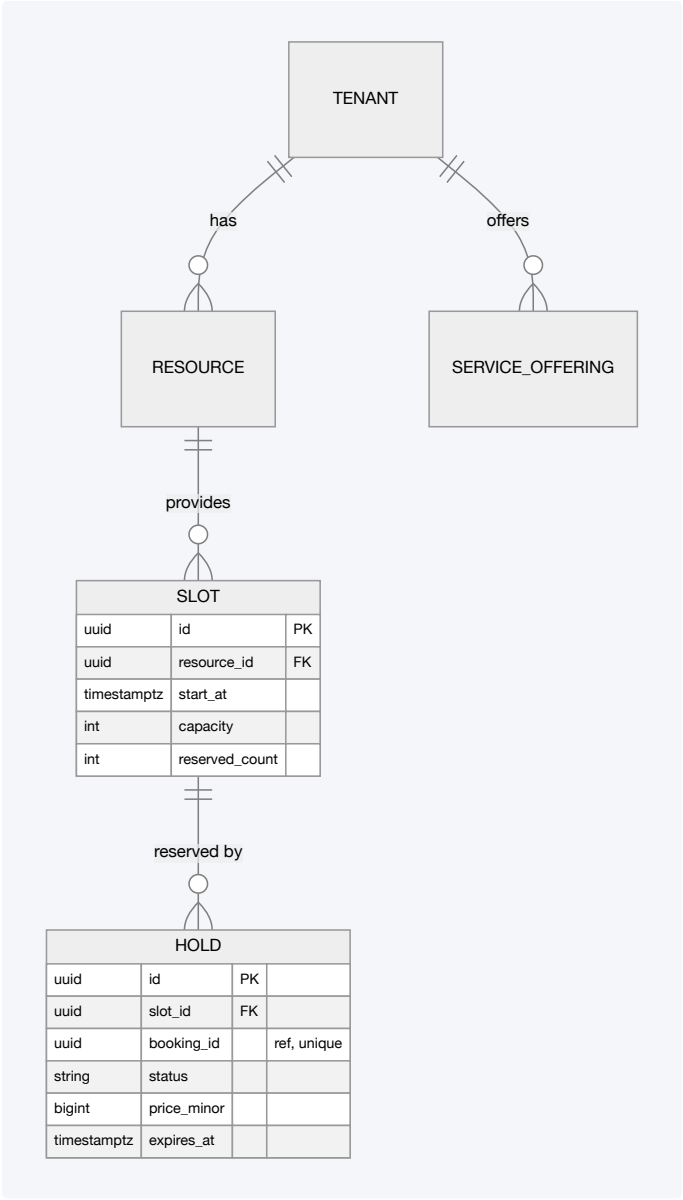
Booking (`BookingStateMachine`)



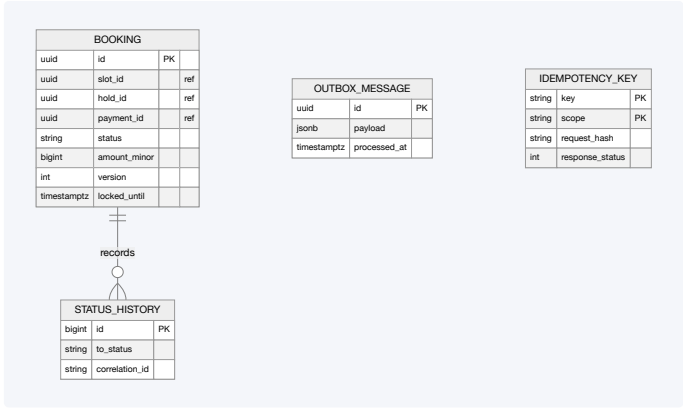
Hold (`Hold`). RELEASED/EXPIRED return capacity once.

Any other transition is rejected with `409 INVALID_STATE_TRANSITION`. Payment: PENDING → SUCCEEDED | FAILED; SUCCEEDED → PARTIALLY_REFUNDED → REFUNDED. Notification message: PENDING → SENT | FAILED_RETRYING → SENT | DEAD.

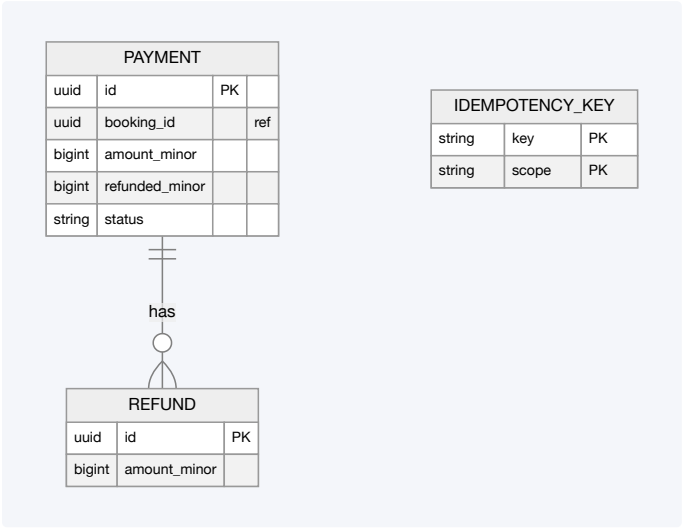
4. Data design



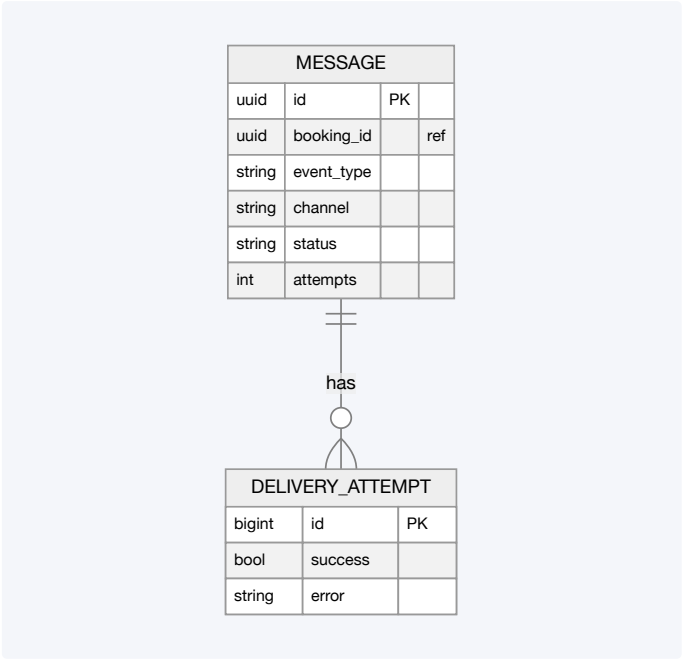
scheduling_db. CHECK 0 ≤ reserved_count ≤ capacity; EXCLUDE (resource_id, tstzrange) prevents overlapping slots.



booking_db (outbox and idempotency keys live with the data they protect).



payment_db. CHECK refunded_minor ≤ amount_minor.



notification_db. UNIQUE (booking_id, event_type, channel) suppresses duplicates.

Ownership and cross-service references. Four databases on one PostgreSQL server locally, each with its own login. The init script revokes `CONNECT` from `PUBLIC`, and we verified that `booking_user` cannot connect to `scheduling_db`. There are no foreign keys across services: references (`slot_id`, `hold_id`, `payment_id`, `booking_id`) are plain UUIDs. Data a service needs at transaction time is copied as a **snapshot**: the hold snapshots the price and slot start, and the booking copies them. Booking can then apply BR5 without calling Scheduling, and a later price change never alters an existing booking. Enums are stored as UPPER_SNAKE_CASE text, the same as in JSON, so database rows read like API responses.

5. API design

Service	Endpoints (all under `/api/v1`)
Scheduling	`POST/GET /tenants`, `POST/GET /tenants/{id}/resources`, `POST/GET /tenants/{id}/offerings`, `POST /slots` (batch), `GET /slots?tenantId&resourceId&from&to&available`, `GET /slots/{id}`, `POST /holds`, `GET /holds/{id}`, `POST /holds/{id}/confirm`, `POST /holds/{id}/release`, `POST /admin/holds/expire-now`
Booking	`POST /bookings*`, `GET /bookings/{id}`, `GET /bookings?customerId&status`, `POST /bookings/{id}/payment*`, `POST /bookings/{id}/cancel*`
Payment	`POST /payments*`, `GET /payments/{id}`, `GET /payments?bookingId`, `POST /payments/{id}/refunds*`, `GET/PUT /admin/chaos`
Notification	`POST /notifications` (202), `GET /notifications/{id}`, `GET /notifications?bookingId`, `GET/PUT /admin/chaos`

* requires `Idempotency-Key`. Status codes: 201 created, 200 OK/replay, 202 accepted, 400 validation, 402 declined, 404 not found, 409 business conflict, 422 key reused with a different payload, 503/504 downstream unavailable/timeout. OpenAPI 3 documents: `docs/openapi/{scheduling,booking,payment,notification}.json` (UI at `/scalar` on each service).

```
POST /api/v1/bookings
Idempotency-Key: 7d9f1c2e-…
X-Correlation-Id: demo-001
{ "tenantId": "01a1…", "slotId": "01a1…",
  "customer": { "id": "c-1001",
                "phone": "+919800000001" } }

201 Created
{ "id": "01a16ce6-720f…", "status": "HELD",
  "holdId": "01a16ce6-7215…", "amountMinor": 50000,
  "currency": "INR",
  "holdExpiresAt": "2026-10-24T07:53:54Z",
  "history": [ {"to":"PENDING"}, {"to":"HELD"} ] }
```

```
409 Conflict   (application/problem+json)
{ "type": "https://slothub.dev/errors/slot-unavailable",
  "title": "Slot unavailable",
  "status": 409,
  "detail": "Slot 01a1… has no remaining capacity.",
  "instance": "/api/v1/holds",
  "code": "SLOT_UNAVAILABLE",
  "correlationId": "c1-20261024T085619Z-7" }

400 → also "errors": { "Name": ["'Name' must not be empty."]
}
```

Error model. All four services return RFC 9457 Problem Details through one shared handler, including validation errors (FluentValidation), malformed JSON and unknown routes. Each response has a stable machine-readable `code` (a catalogue in `ErrorCodes`) and the `correlationId`. Downstream errors keep their code when Booking passes them on: `SLOT_UNAVAILABLE` from Scheduling reaches the client unchanged.

6. Engineering challenges

Challenge 1: Concurrency control (BR1)

Problem. Many customers try to hold the last unit of a slot at the same instant. **Design.** Check and increment are one SQL statement: `UPDATE slots SET reserved_count = reserved_count + 1 WHERE id = @slot AND reserved_count < capacity AND start_at > now()`. PostgreSQL row-locks the slot during the update, so concurrent requests queue. Each re-evaluates the `WHERE` against the committed value, and 0 rows affected means `409 SLOT_UNAVAILABLE`. The hold insert is in the same transaction, and a CHECK constraint is a second line of defence. **Why check-then-update fails:** with `if (slot.Reserved < slot.Capacity) { slot.Reserved++; Save(); }`, every request reads 0, every request passes the check, and every request writes 1. We built that version behind `CONCURRENCY_MODE=naive` to show it.

Evidence (`evidence/challenge-1-concurrency/`)	Result
20 simultaneous requests, capacity 1 (FIFO barrier releases all at once)	1 × 201, 19 × 409 SLOT_UNAVAILABLE; reserved_count = 1; one HELD row
30 simultaneous requests, capacity 5	5 × 201, 25 × 409; reserved_count = 5
Same 20 requests against the naive instance	**20 holds for capacity 1**, reserved_count = 1 (lost updates)
Integration test, real PostgreSQL: 20 and 50 parallel reservations	exactly capacity reserved

Alternatives. `SELECT … FOR UPDATE` then update: correct, but two round trips and a longer lock. Optimistic version column: correct, but most requests fail and retry under contention. One row per seat with a unique constraint: right for numbered seats, overkill for counts. Distributed lock (Redis): extra infrastructure when the database is already the source of truth. **Limitation:** one very hot slot is bounded by row-lock throughput.

Challenge 2: Idempotency (BR4)

Design. A shared middleware handles endpoints marked unsafe. It (1) fingerprints the request: SHA-256 of method, path and canonical JSON, so key order and whitespace do not matter. (2) It runs `INSERT … ON CONFLICT DO NOTHING` on `(key, scope)` in the service's own `idempotency_keys` table, so the unique key *is* the lock. Two identical requests at the same moment cannot both win. (3) The winner runs, and its status and body are stored. A repeat is replayed with `Idempotency-Replayed: true`. A different fingerprint gets 422. A request still in progress gets `409 REQUEST_IN_PROGRESS` with `Retry-After`. A 5xx releases the key, so the client may retry. Booking calls Payment with **derived keys** `pay-{bookingId}` and `refund-{bookingId}`, so even Booking's own retries cannot double-charge. Payment finishes an in-flight charge even if the caller disconnects: we found that otherwise an empty body was stored and replayed.

Evidence (`evidence/challenge-2-idempotency/`, 20/20 checks)	Result
Same key + body ×3	All 201; 2nd and 3rd replayed with an identical body hash; 1 booking row
Same key, different body / no key	422 IDEMPOTENCY_KEY_MISMATCH / 400 IDEMPOTENCY_KEY_REQUIRED; no row created
10 identical POST /bookings at the same instant	1 × 201, 9 × 409 REQUEST_IN_PROGRESS; 1 booking, 1 hold
10 identical POST /payment at the same instant; repeat; new key	1 payment row, 50 000 charged once; replay; still one charge
Integration tests (API + PostgreSQL), 20 simultaneous charges	exactly one payment; 5xx releases the key

Alternatives. In-memory cache: lost on restart, not shared across replicas. Redis: extra infrastructure, and it separates the key from the data it protects. A unique business constraint alone (e.g. one payment per booking) does not return the original response. **Limitation:** keys are scoped per path, not per client (no authentication). They are deleted after 24 h.

Challenge 3B: Time-bound holds (BR2), with BR3 compensation

Design. Each hold has `expires_at = now + TTL`. A background worker (every 5 s, also exposed as `POST /admin/holds/expire-now`) claims due holds with `SELECT … WHERE status = 'HELD' AND expires_at <= now FOR UPDATE SKIP LOCKED`. It applies the domain rule `Hold.Expire()` and returns capacity in the same transaction. **Safe to run twice:** only HELD rows qualify, so a second run finds nothing. **Never releases a paid slot:** confirm takes the row lock and requires `expires_at > now`, so confirm and expiry race on one row and exactly one wins. **Multi-replica safe:** SKIP LOCKED lets several sweepers split the work. Booking treats Scheduling as the source of truth: its reconciler asks for the hold before marking a booking EXPIRED, so two clocks can never disagree. If payment lands after expiry, the confirm fails and Booking refunds automatically.

Evidence (`evidence/challenge-3b-holds/`, 18/18 checks, TTL 30 s)	Result
Unpaid hold	EXPIRED by the worker; capacity 1 → 0; booking EXPIRED; BOOKING_EXPIRED notification
Sweep run twice	Second run expires 0; capacity not released twice
Paid hold after TTL + sweeps	Still CONFIRMED, capacity still 1
Cancel while HELD	RELEASED immediately, capacity 0
Payment started 26 s in, gateway 6 s slow	Attempt timeout, retries poll REQUEST_IN_PROGRESS, stored charge replayed; confirm → HOLD_EXPIRED; refunded; booking EXPIRED
Integration: 4 concurrent sweepers over 10 due holds	Each hold expired exactly once; capacity 0

Alternatives. A per-hold timer in memory: lost on restart, not shared. A broker's delayed messages: a new dependency, and still needs an idempotent consumer. Database TTL/cron: same SQL, less visible. **Limitations:** expiry granularity is the sweep interval (a hold can live up to TTL + 5 s; confirmation enforces the exact TTL). A refund that fails after a late payment is only logged.

> **Beyond the required challenges.** Booking's calls to Scheduling and Payment use timeouts (2 s per attempt), retries with exponential backoff and jitter, and a circuit breaker (Polly). The e2e script shows the circuit opening during a gateway outage (`DOWNSTREAM_CIRCUIT_OPEN`) and closing after 15 s; the retry with the same key then charges exactly once. This is the Challenge 3A bonus behaviour, kept safe by Challenge 2.

7. Testing

Level	What	Result
Unit (xUnit)	Business rules named `BRn_…`, state machines, refund policy, fingerprinting, retry backoff, templates; Booking orchestration with in-memory fakes (declined, gateway down, late payment, lease busy, reconciliation)	92 pass (Scheduling 24, Booking 42, Payment 8, Notification 13, BuildingBlocks 5)
Integration (Testcontainers, real PostgreSQL)	Atomic hold under 20/50-way concurrency; expiry run twice, confirmed untouched, 4 concurrent sweepers; Payment API idempotency (replay, 422, 400, 20 simultaneous, 5xx release)	9 pass
End-to-end script	Hold → pay → confirm → SMS + email; declined → FAILED, capacity restored; gateway down → 503, HELD, circuit open, recovery, one charge	26/26 checks
Challenge scripts	§6	C1 PASS, C2 20/20, C3B 18/18
Postman (newman)	38 requests covering happy path and errors	71/71 assertions
Fresh-copy check	Clean copy of the repository on a new database volume, following only the README	All of the above pass

CI (GitHub Actions) restores, builds with warnings as errors and runs all tests on every push and PR. `scripts/run-all-checks.sh` regenerates all evidence in about three minutes. The tests found real defects during development: the empty replay body after a client timeout, the circuit breaker needing a recovery window, and two script bugs.

8. Design decisions and trade-offs

#	Decision	Options	Choice and rationale
D1	Number of services	5 (separate Catalog) vs 4	**4.** Catalog and availability change together; one fewer hop; can be split later if catalog reads dominate.
D2	Coordination	Orchestration vs choreography	**Orchestration in Booking.** The flow and its compensation (release, refund) are explicit, in one place and unit-testable. Choreography would need a broker and scatter the logic.
D3	Communication	Sync REST vs async messaging	**Sync REST** where the customer waits (hold, pay, confirm); **outbox + async dispatch** for notifications so an SMS outage never blocks or undoes a booking. The outbox maps directly to a broker later.
D4	Concurrency	Pessimistic lock, optimistic version, atomic conditional update	**Atomic conditional update** for the contended slot (one statement, shortest lock, no retry storm); row locks for low-contention hold/payment updates.
D5	Idempotency storage	Memory, Redis, own DB table	**Own DB table per service.** Survives restarts, works across replicas, keeps database-per-service, no new infrastructure.
D6	Checkout shape	One-shot vs hold-then-pay	**Hold-then-pay.** Mirrors real payment pages, gives holds a real lifetime, and makes the late-payment case visible and handled.
D7	Serialising work on one booking	DB lock across HTTP calls, version only, lease	**Short lease (`locked_until`) + version.** No DB lock is held while calling other services; a crashed operation's lease simply expires.
D8	Multi-tenancy	DB per tenant vs `tenant_id` column	**`tenant_id` on every row.** Cheapest to run; large tenants can move to dedicated databases later.

9. Limitations and future work

- No authentication: `tenantId` and `customerId` are trusted from the request. Next step: OIDC with tenant claims at an API gateway.
- Synchronous orchestration: if Scheduling or Payment is down, new bookings fail fast (503) rather than queue. Timeouts, retries and the circuit breaker bound the impact.
- A refund that fails after a late payment is only logged (manual follow-up); next: refunds through the outbox. Retrying a cancellation across the 24 h boundary computes a different refund and gets 422.
- Slots are created explicitly; recurring schedules, waitlists, memberships and discounts are planned as new services, not rewrites.
- One PostgreSQL server hosts the four databases locally (separate logins). Separate instances, a broker (RabbitMQ via the outbox) and OpenTelemetry tracing are the next scaling steps.

10. Links

Google Drive (video, report, other documents)	Main SlotHub Google Drive folder
Combined team demo (Subhodip → Shekar → Ria)	Watch combined demonstration
Individual demonstration videos	J K Shekar: Watch demonstration, Ria Mishra: Watch demonstration, Subhodip Banerjee: Watch demonstration
Source code	github.com/2025tm93191-stack/slothub — README, `docs/`, `evidence/`, `postman/`

Appendix A — Contribution table

Member (ID)	Owned	Reviewed	Report / video parts	Share
Subhodip Banerjee (2025TM93191), 20 commits	Architecture and backlog, BuildingBlocks (errors, logging, correlation, idempotency), Scheduling (catalog, atomic holds C1, expiry C3B), Booking orchestration (outbox, refunds), Testcontainers tests, scripts, CI, report	Payment, Notification	§1–3, §6 C1/C3B, §8	33⅓%
J K Shekar (2025TM93095), 27 commits	Payment domain and contracts, Notification service and templates, Payment/Notification unit and integration tests, C2 and C3B evidence, test screenshots, architecture doc updates	Booking, Scheduling	§4, §6 C2, §7	33⅓%
Ria Mishra (2025TM93010), 18 commits	Payment service: entity, mock gateway, repository, endpoints, migration, Dockerfile, payment and idempotency tests	Notification	§5	33⅓%
Harshit Pathak (2025TM93557)	—	—	—	0%

Appendix B — Individual reflections

Half a page each: what I built, the hardest problem I solved, what I would change.

Subhodip Banerjee (2025TM93191). *What I built:* the architecture and backlog, the shared building blocks (errors, logging, correlation ID, idempotency middleware), the Scheduling service with its atomic holds and expiry worker, and the Booking orchestration with the outbox and refunds. *Hardest problem:* making the slot capacity safe under concurrency. My first instinct was to read the slot, check `reserved_count < capacity` in C# and then insert the hold, but with 50 parallel requests that read-then-write lets several requests see the same free seat. Moving the check into one conditional `UPDATE … WHERE reserved_count < capacity`, in the same transaction as the hold insert, turned the race into a row count of 0 or 1. I kept a naive mode behind a flag so the overbooking is visible in the demo. The second hard part was in Booking: pay, cancel and the expiry reconciler could act on the same booking at once. A per-booking lease (`locked_until`) serialises them, and writing the notification to an outbox in the same transaction means a crash cannot confirm a booking without notifying, or notify without confirming. *What I would change:* replace the polling outbox dispatcher and reconciler with a message broker so that services react to events instead of polling, and add OpenTelemetry tracing so a booking can be followed across services in one view instead of by correlation ID in logs.

J K Shekar (2025TM93095). *What I built:* the Payment domain model and service contracts, pagination and partial refunds, the Notification service with its templates (including the booking reminder), the unit and integration tests for both services, and the evidence runs for Challenges 2 and 3B. *Hardest problem:* the Challenge 2 script sometimes hung during the concurrent idempotency test. It started the parallel requests in the background and then called a bare `wait`, which waits for *every* background job of the shell, not only the requests. Collecting the worker PIDs and waiting on those alone fixed it, and I applied the same fix to the Challenge 1 script, which had the same bug. A similar timing issue made the Challenge 3B late-payment scenario flaky: it slept a fixed number of seconds after creating the hold, so on a slow machine the payment did not always finish after expiry. I changed it to read `holdExpiresAt` from the response, start paying 6 s before it and make the mock gateway take 9 s, so the payment always lands after the hold has expired. *What I would change:* move these timing scenarios into integration tests with a controllable clock instead of real sleeps, and add contract tests between Booking and Payment so an API change is caught before deployment.

Ria Mishra (2025TM93010). *What I built:* the core of the Payment service: the payment entity, the mock payment gateway, the repository, the HTTP endpoints, the initial EF Core migration and the Dockerfile, together with the payment unit tests and the idempotency integration tests. *Hardest problem:* making a charge happen exactly once. Booking may retry a charge after a timeout, and the client may retry too, so the same request can arrive twice or at the same moment. The idempotency tests prove each case against a real PostgreSQL container: a repeated key replays the original response without a second charge, the same key with a different body returns 422, two simultaneous identical requests charge once, and a 5xx releases the key so the retry can succeed. The mock gateway reads the chaos settings, so declines (402) and gateway outages (503) can be triggered to test Booking's error handling. *What I would change:* replace the mock with a sandbox of a real gateway behind the same interface, handle asynchronous gateway callbacks (webhooks) instead of assuming the charge result is immediate, and add a daily reconciliation job between our payment records and the gateway's.

Harshit Pathak (2025TM93557). No contribution.

Appendix C — AI-use declaration

Tool	Used for
Claude (Anthropic), via Claude Code	Domain brainstorming and service decomposition options; drafting the architecture document and backlog; generating most of the C# code, EF Core migrations, tests, scripts, Postman collection and this report's first draft; diagnosing failures found by the tests.

No other AI tools were used.

Every member reviewed, ran and can explain the code they present in the demo. All design decisions in §8 were discussed and accepted by the team.

Appendix D — Business-rule traceability

BR	Code	Tests and evidence
BR1	`Scheduling.Infrastructure/Persistence/Repositories.cs` (`HoldRepository.ReserveAsync`) + CHECK constraint	`HoldConcurrencyTests.BR1_ParallelHolds_NeverExceedCapacity`, `BookingServiceTests.BR1_*`, challenge1 scripts
BR2	`Scheduling.Domain/Holds/Hold.cs` (`Confirm`, `Expire`), `HoldRepository.ExpireDueAsync`, `HoldExpiryWorker`	`HoldTests.BR2_*` (7), `HoldConcurrencyTests.BR2_*` (2), challenge3b script
BR3	`Booking.Domain/Bookings/Booking.cs` (`EnsurePayable`, `Confirm`), `BookingService.PayAsync`	`BookingTests.BR3_*`, `BookingServiceTests.BR3_*`, challenge3b section E
BR4	`shared/BuildingBlocks/Idempotency/*`, derived keys in `Booking.Infrastructure/Clients/PaymentClient.cs`	`RequestFingerprintTests.BR4_*`, `Payment.IntegrationTests/IdempotencyTests.BR4_*`, challenge2 script
BR5	`Booking.Domain/Bookings/RefundPolicy.cs`, `BookingService.CancelAsync`	`RefundPolicyTests.BR5_*`, `BookingServiceTests.BR5_*`, Postman "Cancel → refund per BR5"
BR6	`Scheduling.Domain/Slots/Slot.cs` (`HasStarted`, `CanBeHeld`), SQL `start_at > now()`	`SlotTests.BR6_*`

Appendix E — How to run

```
docker compose up -d --build --wait     # PostgreSQL (4 DBs, 4 users) + 4 services on localhost:5101–5104
./scripts/run-all-checks.sh             # tests, e2e, Challenges 1/2/3B, Postman, logs → evidence/
dotnet test SlotHub.slnx                # 101 tests (Docker needed for the integration tests)
```

Full instructions, every environment variable and troubleshooting: `README.md`. Evidence index: `evidence/README.md`.